

Docker'ı tanıyalım

Şirket, platform ve komutların arkasındaki görevler.



Docker, Inc.

Ürünleri geliştiren şirket.
dotCloud → Docker, Inc. (2013)
Kurucu: Solomon Hykes.

Docker platformu

Uygulamaları image olarak hazırlar; paylaşmayı ve container olarak çalıştırmayı kolaylaştırır.

Komutu sen verirsin; işi Engine yönetir.



CLI

Komutu API isteğine çevirir.

API isteği



Engine / server



Image ve container işlemlerini yönetir. Daemon: dockerd.

image

container

CLI ve Engine aynı makinede veya ağ üzerinden farklı makinelerde olabilir.

İletişimi doğrula

YALNIZCA CLI SÜRÜMÜ

```
docker --version
```

CLIENT + SERVER BİLGİLERİ

```
docker version
```

Client var, Server'a erişilemiyor: istemci kurulu; bağlantı veya servis kontrolü gerekir.

Desktop ve Engine aynı rol değildir.



Docker Desktop: Engine + CLI + arayüz + geliştirme araçları.



Docker Engine: container işlemlerinin server tarafı. Terminal, çalışan uygulamanın kendisi değildir.

Ortak kurallar, güçlü ekosistem

OCI standart koyar. CNCF projeleri destekler. Moby yapı taşları sunar.

OCI • birlikte çalışabilmenin kuralları



IMAGE SPEC

Image'ın dosya ve yapılandırma biçimi.



RUNTIME SPEC

Container ortamının çalıştırılma kuralları.



DISTRIBUTION SPEC

Image içeriğinin registry ile aktarım kuralları.

CNCF • projelerin geliştiği ortam

Topluluk, yönetim, dokümantasyon ve gelişim desteği sağlar.

Sandbox

Erken gelişim; topluluk ve yapı.

Incubating

Artan benimsenme; olgunlaşan süreçler.

Graduated

Daha güçlü olgunluk ölçütleri karşılanmış.

Örnek projeler: Kubernetes • containerd • Prometheus • Cilium

Olgunluk seviyesi bir sinyaldir; her kullanım için uygunluk garantisi değildir.

Moby • platform oluşturmak için bileşenler



Build araçları



Runtime



Registry



Diğer araçlar

Moby ve diğer projelerin bileşenleriyle container sistemleri kurulabilir.

Tarihçe ve güncel ayırım

appc / rkt rekabetinden ortak OCI standartlarına geçildi. Kitaptaki Notary v1, 2025'te arşivlendi; modern Notary Project / Notation ile karıştırma.

Docker nerede çalışıyor?

Host terminali, Linux ortamı ve container birbirinden farklıdır.

Docker Desktop • Mac / Windows / Linux

Host CLI



Terminalindeki docker



Linux ortamı / VM

Engine

container A

container B

PAYLAŞILAN LINUX KERNEL'İ

Mac: Client darwin/arm64, Server linux/arm64 olabilir; bu normaldir.

Multipass • VM'yi sen oluşturursun

HOST TERMINALİNDE • GÜNCEL YOL

```
multipass launch 24.04 --name node1
multipass ls
multipass shell node1
```

ARDINDAN VM İÇİNDE • KİTABIN UBUNTU SNAP YOLU

```
sudo snap install docker
sudo docker version
```

2026: Docker blueprint, Multipass 1.17'de kaldırıldı. VM'ye Docker ayrıca kurulur.

Doğrudan Linux kurulumu ve kullanıcı izni

Engine Linux host üzerinde çalışır; container'lar host kernel'ını paylaşır. Kitap Canonical Snap'i gösterir; Docker'ın resmî APT yolu ayrı bir kurulum seçeneğidir.

docker grubu ≈ root düzeyi yetki

Grup üyeliği yeni oturuma yansımalıdır; CLI kurulumu erişim iznini kanıtlamaz.

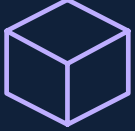
Doğrulama ve servis ayrımı

docker version → Client + Server. Kitaptaki service ... start, restart değildir. Snap servisleri snap komutlarıyla yönetilir.

Image: uygulamanın hazır paketi

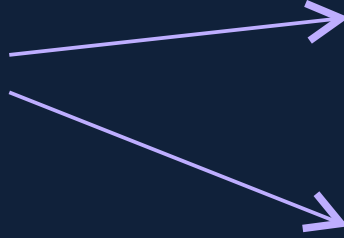
Dosyalar, bağımlılıklar ve başlangıç ayarları; çalışan süreçten farklı bir nesne.

Aynı image'dan birden çok container



IMAGE

Salt okunur katmanlar



Container A

Ayrı süreç + yazılabilir katman

Container B

Ayrı süreç + yazılabilir katman

REGISTRY'DEN YEREL IMAGE DEPOSUNA

```
docker pull nginx:latest
```

pull image'ı alır; uygulamayı başlatmaz. Eksik katmanlar indirilir; mevcut katmanlar yeniden kullanılabilir.

Bir image referansını oku

```
docker.io/library/nginx:latest
```

REGISTRY

docker.io

NAMESPACE

library

REPOSITORY

nginx

TAG

latest

docker images → bağlı olduğun Engine'in yerel image listesi.

Tag, digest ve Image ID

Tag değişebilir; digest belirli içeriği seçer. Image ID ve registry digest'i farklı nesnelerin kimlikleri olabilir.

Çıktıyı yorumla; sayıları ezberleme.

CREATED: image oluşturulma zamanı. SIZE: indirme trafiği değildir; katmanlar paylaşılabilir.

NGINX'i çalıştır, portu bağla

Image'dan yeni bir container: isim, ana süreç ve host portu.

HOST / VM'DE DOCKER CLI KULLANDIĞIN TERMİNAL

```
docker run --name test -d -p 8080:80 nginx:latest
```

--name test

Container'ın adı.

-d

Terminal bağlı kalmaz.

nginx:latest

Kullanılacak image.

8080:80 • iki sayı, iki farklı taraf


HOST
8080

PORT EŞLEMESİ



test
NGINX
80

Sağdaki portu uygulamanın dinlediği port belirler; eşleme onu değiştirmez.

Çalışıyor mu? docker ps ile bak.

IMAGE	nginx:latest
NAMES	test
STATUS	Up ...
PORTS	0.0.0.0:8080->80/tcp

-d uygulamanın yaşamını uzatmaz.

NGINX ana süreç olarak çalışır. Ana süreç kapanırsa container durur. "Up" tek başına HTTP cevabının doğru olduğunu kanıtlamaz.

İçeri gir; süreçleri ve yaşamı ayır

exec yeni süreç açar. exit, stop ve rm farklı nesneleri etkiler.

HOST / VM TERMİNALİ • test ÇALIŞIYOR OLMALI

```
docker exec -it test bash
```

Var olan test container'ı

NGINX • ana süreç

Çalışmaya devam eder.

Bash • ek süreç

exec ile başlatılır.

-i: klavye girdisi açık. -t: sanal terminal. Yeni container oluşturulmaz.

Bu komutlar artık container içindeki Bash'te:

```
ls -l
ps -elf
```

ps: command not found

ps aracı yoktur. Bu mesaj, NGINX'in çalışmadığını göstermez.

Aynı lab, üç ayrı etki

1 exit

Yalnızca ek Bash kapanır; test çalışır.

2 docker stop test

Ana süreç durur; container kaydı kalır.

3 docker rm test

Container ve yazılabilir katmanı kaldırılır.

exit: container kabuğunda. stop / rm: geri döndüğün host veya VM terminalinde.

Sonucu doğru listede kontrol et.

docker ps: yalnızca çalışan container'lar.

docker ps -a: durmuş olanlar da dâhil.

docker images: nginx:latest hâlâ yerelde kalabilir.

Koddan image'a: Dockerfile

Tarifi okumak, image oluşturmak ve uygulamayı başlatmak ayrı adımlardır.

PROJE DOSYALARINI AL • git CLI GEREKLİ

```
git clone https://github.com/nigelpoulton/psweb.git
cd psweb
```

cat Dockerfile • KİTAPTAKİ SEKİZ TALİMAT

```
FROM alpine
LABEL maintainer="nigelpoulton@hotmail.com"
RUN apk add --update nodejs npm curl
COPY . /src
WORKDIR /src
RUN npm install
EXPOSE 8080
ENTRYPOINT ["node", "./app.js"]
```

Her satırın görevi

FROM	Alpine taban image'ından başla.
LABEL	Tanımlayıcı bilgiyi image'a kaydet.
RUN apk ...	Build sırasında Node.js, npm ve curl kur.
COPY . /src	Bağlamdaki dosyaları image'ın /src dizinine kopyala.
WORKDIR	Sonraki işlemlerin çalışma dizinini /src yap.
RUN npm ...	Build sırasında uygulama bağımlılıklarını kur.
EXPOSE	Beklenen container portunu belirt; host'a yayımlamaz.
ENTRYPOINT	Container başlayınca node ./app.js çalıştır.

psweb İÇİNDE • SONUNDAKİ NOKTAYI UNUTMA

```
docker build -t test:latest .
```

-t: image etiketi | .: build bağlamı

Builder'ın erişebildiği dosyalar bağlamdan gelir. Buildx isteği iletir; BuildKit image'ı hazırlar. docker images ile test:latest kaydını doğrula.

Uygulamayı yayımla, sonucu doğrula

Image → çalışan container → HTTP cevabı: üç ayrı kontrol.

YENİ CONTAINER: web1 • IMAGE: test:latest

```
docker run -d --name web1 --publish 8080:8080 test:latest
```

İstek uygulamaya nasıl ulaşır?

HOST PORTU

8080



CONTAINER PORTU

8080



Node / app.js

docker ps → web1 / Up ... • Tarayıcı → uygulamanın gerçek cevabı

Docker Desktop



http://localhost:8080

Desktop, host bağlantısını Linux ortamındaki container'a iletir.

Multipass

```
multipass ls
```

http://VM_IP:8080

VM'nin gerçek IPv4 adresini kullan. Her IP adresi 192 ile başlamaz.

Kitabın temizleme sırası

1 `docker rm web1 -f`

Çalışan container'ı SIGKILL ile sonlandırıp kaldırır. Image kalır.

2 `docker rmi test:latest`

Yerel image referansını kaldırır; uygun koşullarda image silinir.

Düzenli kapanış için önce stop, sonra rm. Kaynak psweb klasörü bu işlemlerle silinmez.

Bütün modeli birleştir.

Kod + Dockerfile → build ile image.

Image → run ile container. exec → container'da ek süreç.

stop → durdur. rm → container kaldır. rmi → image kaldır.

Son kontrol: `docker ps -a` ve `docker images`